Lab 1 — Digital vs Analog

Time: ~25 minutes (in the Telecom & Data-Centre Edition, this lab is a 3-minute trainer demo)
You'll learn: the difference between a digital signal (on/off) and an analog signal (a range of values), by controlling and reading three parts.

> **Telecom & Data-Centre Edition**
>
> **The story:** The LED is a tower site's status light, the button is the shelter's door contact, and the dial is an analog reading such as battery voltage.
>
> **In the workshop:** The trainer runs this lab on the projector in Module 3. Try it yourself afterwards; it takes about 25 minutes.

What you're building

- An **LED** that blinks on its own (a digital *output* — the code turns it fully on, then fully off, nothing in between).
- A **push button** the code reads (a digital *input* — it's either pressed or not, nothing in between).
- A **potentiometer** (a dial/knob) the code reads (an analog *input* — it reports a whole range of values, not just two).

Wiring diagram

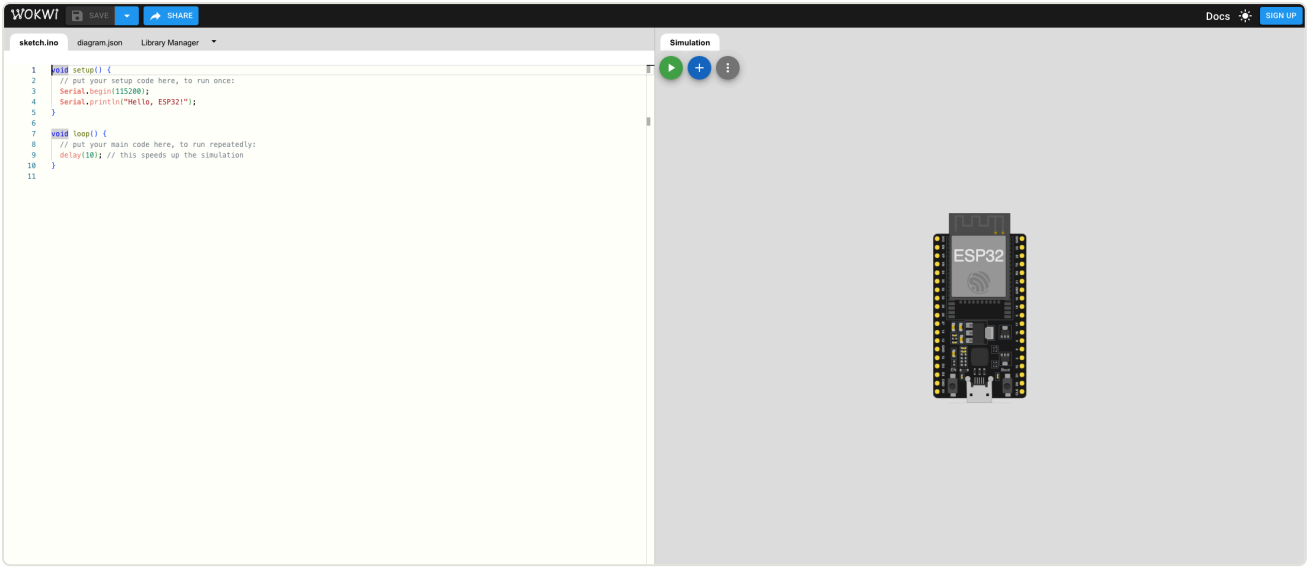
ESP32 Pin	Wire Color	Connects To
`3V3`	red	Potentiometer (VCC)
`GND`	black	Green LED (Cathode), Push Button (leg 2), Potentiometer (GND)
`2`	green	Resistor → LED (Anode)
`15`	yellow	Push Button (leg 1)
`34`	blue	Potentiometer (signal)

You don’t have to wire any of this by hand — `diagram.json` already describes it. You’re just pasting it into Wokwi, which draws the circuit for you exactly as shown above.

Step-by-step

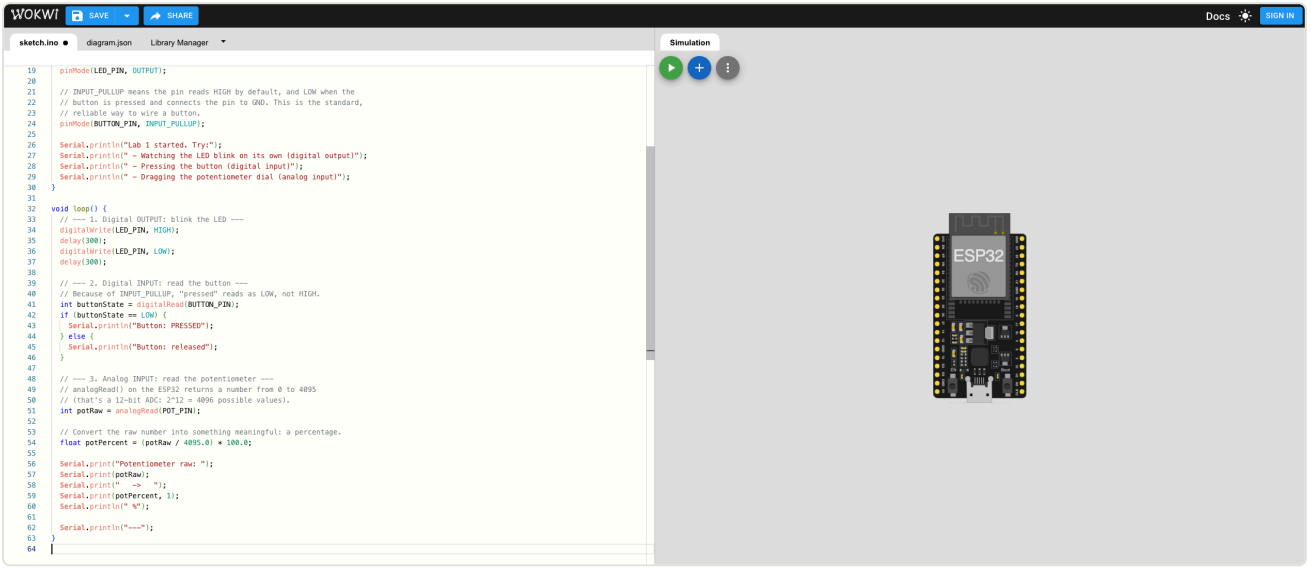
1. Open a new ESP32 project

Go to wokwi.com/projects/new/esp32. You'll see a blank project with a default "Hello, ESP32!" sketch already loaded and a bare ESP32 board on the right.



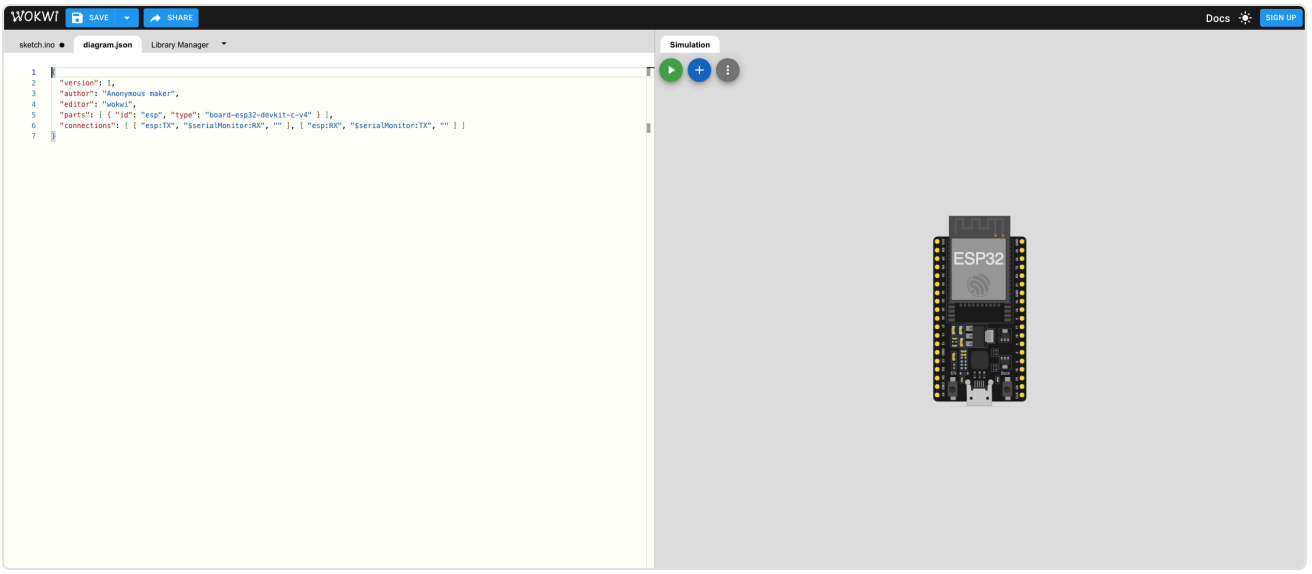
2. Paste the code

Open `sketch.ino` in this folder, select all, copy it. Back in Wokwi, click into the **sketch.ino** tab on the left, select all the default code (`Cmd/Ctrl+A`), and paste over it.

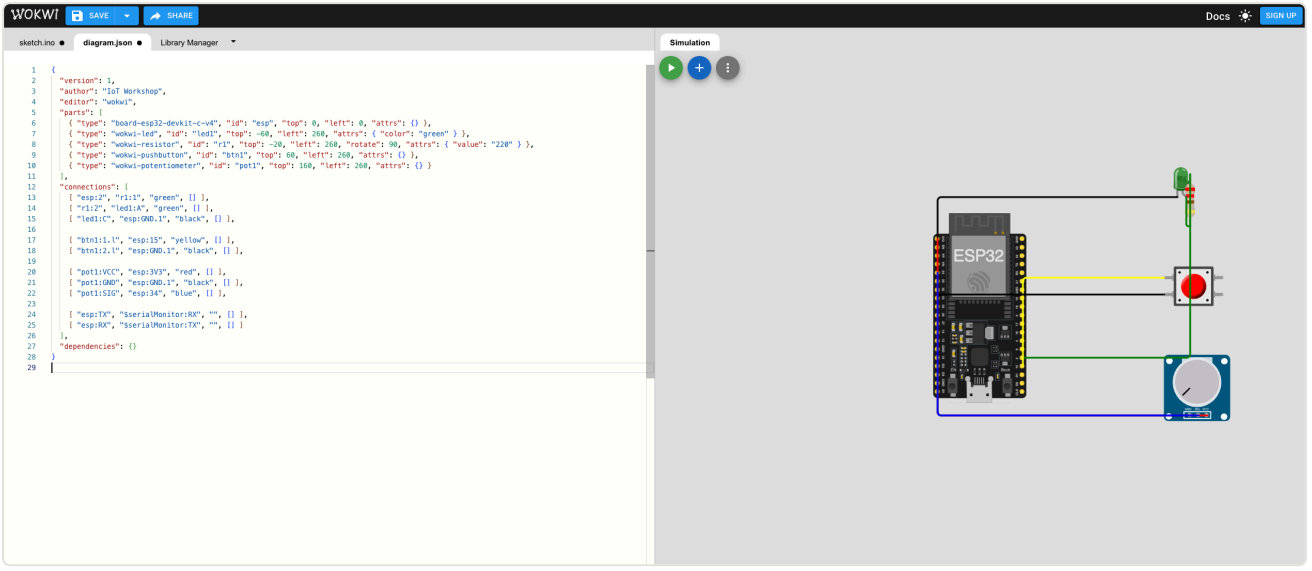


3. Paste the circuit

Click the **diagram.json** tab. It holds a default circuit with just the bare ESP32 board.

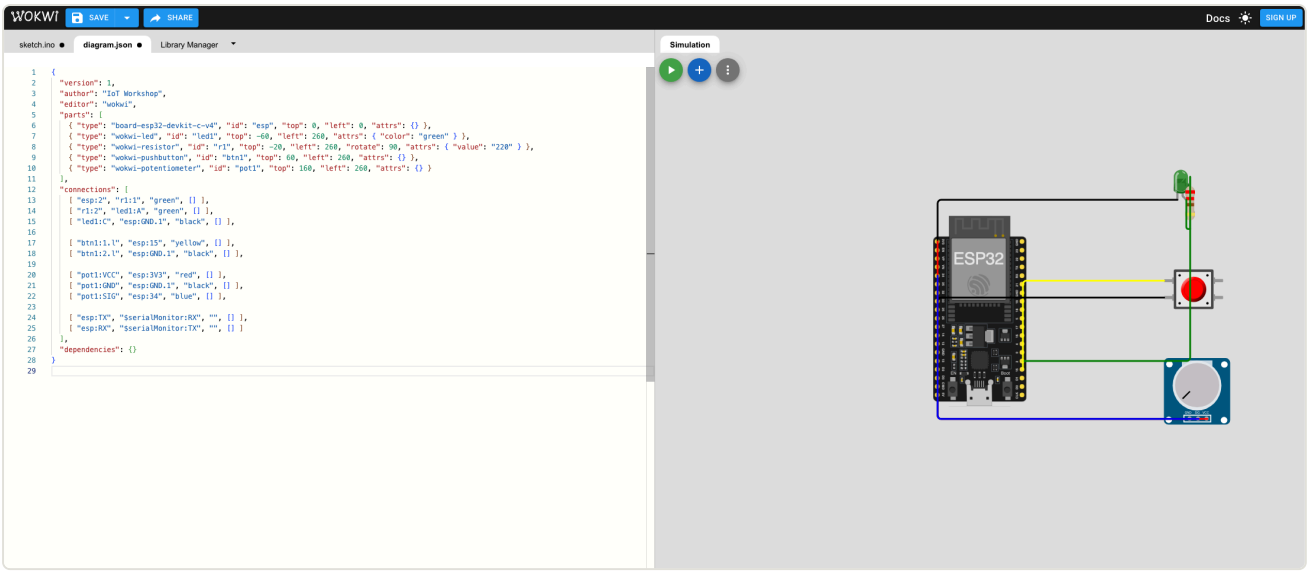


Open `diagram.json` in this folder, select all, copy it, then select all in Wokwi's diagram.json editor and paste over it. The breadboard view on the right updates immediately — you should see the ESP32, LED, resistor, push button, and potentiometer appear, already wired.



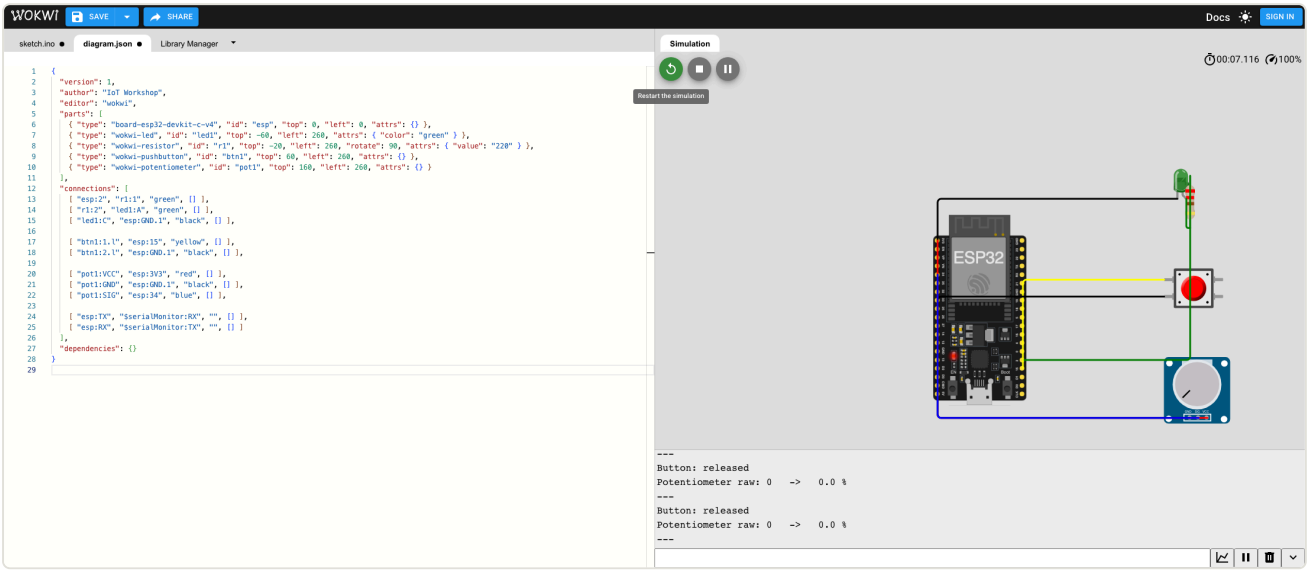
4. Press Play

Click the green ▶ **Play** button (top-left of the simulation pane). The first compile of a fresh project takes a few seconds — Wokwi is genuinely compiling real C++ code on their servers, not just loading a canned demo.



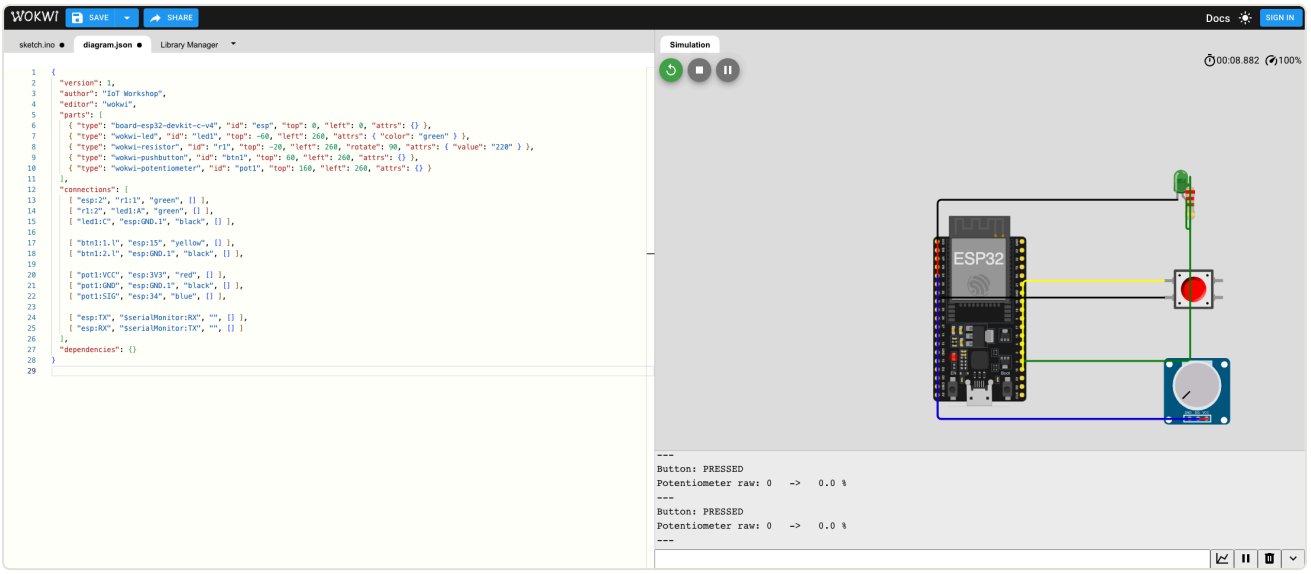
5. Watch it run

Once compiled, the simulation starts automatically, and a **Serial Monitor** panel appears at the bottom of the simulation pane on its own — that's where every `Serial.print()` in the code shows up as text.

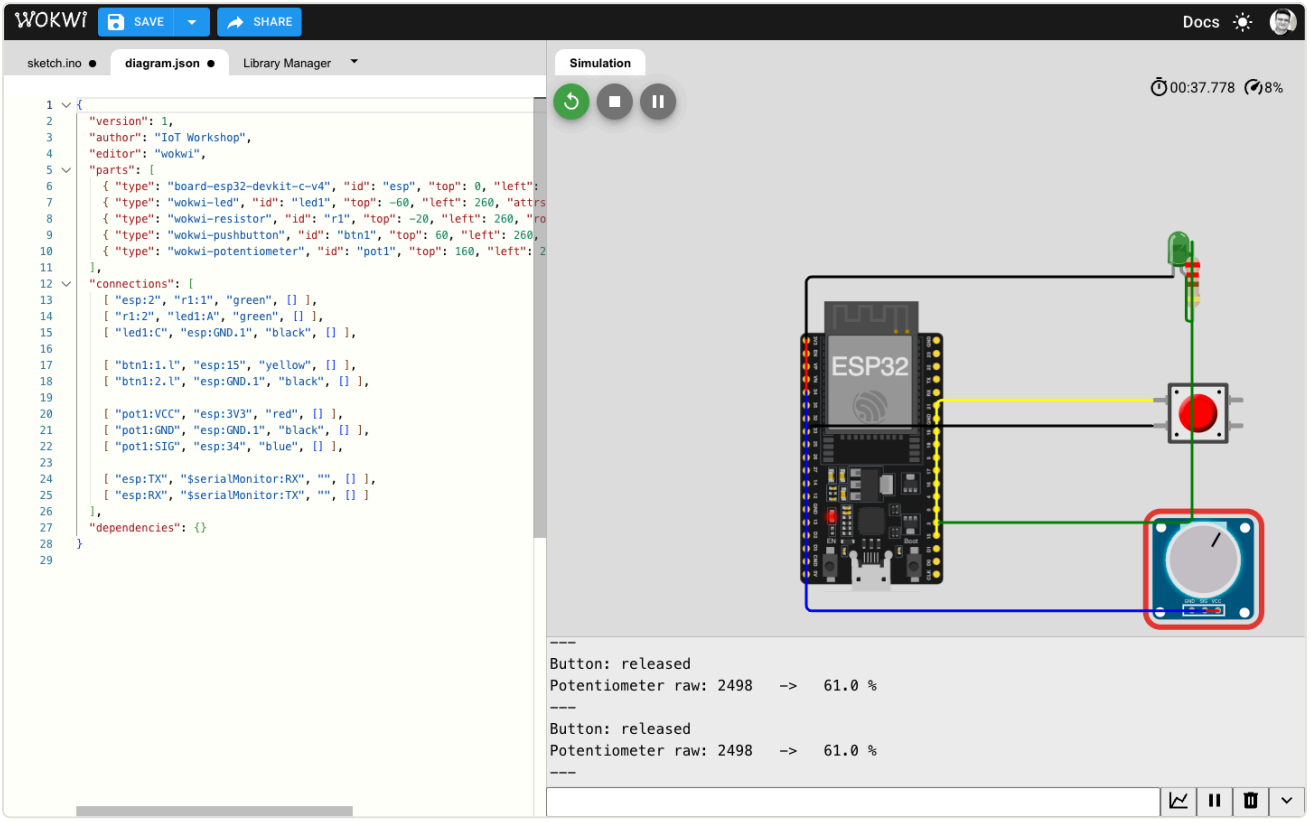


Try it

- **Watch the LED** blink by itself in the diagram view — that's your first digital output. It's either fully on or fully off; there's no "half-lit."
- **Click and hold the push button** in the diagram while the simulation is running. Watch the Serial Monitor print `Button: PRESSED`.



- **Click and drag the potentiometer's dial**. Watch the number in the Serial Monitor change smoothly across the whole range — that's the difference between digital and analog, live. (You can also click the dial once and use the arrow keys, or Page Up and Page Down for bigger steps.)



What you should see

The Serial Monitor prints a startup banner once, then a new reading every loop:

```
Lab 1 started. Try:
 - Watching the LED blink on its own (digital output)
 - Pressing the button (digital input)
 - Dragging the potentiometer dial (analog input)
Button: released
Potentiometer raw: 0   ->   0.0 %
---
Button: PRESSED
Potentiometer raw: 0   ->   0.0 %
---
```

The potentiometer starts at `0` until you drag its dial — that's expected, not a bug.

Understand the code

- `pinMode(LED_PIN, OUTPUT)` tells the chip "I'm going to control this pin."
- `pinMode(BUTTON_PIN, INPUT_PULLUP)` tells the chip "I'm going to read this pin, and by default treat it as HIGH unless something pulls it LOW." This is the standard, reliable way to wire a button — it avoids a common beginner bug where an unpressed button gives random, flickering readings.
- `analogRead()` gives a number from 0 to 4095 on the ESP32 (not 0–1023 like some older boards) — that's because the ESP32's analog-to-digital converter (ADC) has 12-bit precision.

Troubleshooting

- **Nothing prints in the Serial Monitor.** Make sure you pasted the *whole* `diagram.json` — it includes two extra lines near the bottom (`$serialMonitor` connections) that are easy to miss if you only copied part of the file. Without them, Wokwi has nothing to show the Serial Monitor panel at all.
- **The circuit looks wrong / parts are missing.** Re-check step 3 — click directly into the diagram.json editor before pasting, not the sketch.ino one. Both tabs look similar at a glance.
- **Button always reads "released."** Make sure you're clicking and holding directly on the button's red cap in the diagram, not just near it.

Challenge (if you finish early)

- Make the LED brightness follow the potentiometer instead of blinking. Hint: replace `digitalWrite` with `analogWrite` (on ESP32, you may need `ledcWrite`-based PWM — ask your trainer, or look up "ESP32 Arduino PWM ledc" for the exact function names for your Wokwi core version).
- Make the LED only blink while the button is held down.